

إزاي تستخرج القانون المدني المصري ثنائي اللغة (egyption-low.pdf) وتطابق الإنجليزي مع العربي

أنسب طريقة للملف ده: تستخرج طبقة النص (text layer) بمكتبة PyMuPDF (الإصدار 1.24 أو أحدث)، خلية خلية جوه صفوف الجدول. بعد كده تعمل normalization للعربي، وتربط كل مادة إنجليزي بالعربي بتاعها عن طريق رقم المادة Article N ↔ مادة (N) بعد تحويل الأرقام الهندية لأرقام ASCII). OCRJ أو Vision LLMJ تلجأ لهم كfallback أو للمراجعة، مش كطريقة أساسية. في مشروعات مفتوحين اشتغلوا على نفس نوع الملف تقريباً (PDF ثنائي اللغة، 170 صفحة)، وقالوا إن فيه نص native وإن الـ OCR مش لازم.

TL;DR

- **الاستخراج:** ابدأ بـ PyMuPDF على طبقة النص، وخذ كل خلية لوحدها بـ clip من حدود الجدول. أحدث إصدارات PyMuPDF بتراجع العربي بالترتيب المنطقي (logical order)، أما extract_text() في pdfplumber فمتسجل عليها إنها بتقلب العربي. لو الملف طلع scanned، استخدم Azure Document Intelligence أو Gemini. في benchmark اسمه KITAB-Bench، Gemini-2.0-Flash جاب 0.13 CER مقابل 0.54 Tesseract. أما AWS Textract فما بيدعمش العربي أصلاً.
- **المطابقة:** ثبت المطابقة على رقم المادة كمفتاح أساسي، مع state machine بتعمل حاجتين: تكمل الصفوف المكسورة بين الصفحات، وتفرّق مواد قانون الإصدار عن مواد القانون نفسه. جوه المادة، الفقرات العربي المرقمة (١)(٢) بتتطابق مع الإنجليزي بـ LaBSE/bge-m3 (زي vebalign/bertalign). وأي زوج نسبة أطواله غريبة أو التشابه بتاعه (cosine) أقل من حوالي 0.8 يروح لمراجعة يدوية.
- **المخرجات:** ملف JSONL فيه سجل لكل مادة، ومعاه: hierarchy باللغتين، مدى الصفحات، is_repealed، alignment_confidence. في RAG اعتبر كل مادة chunk واحد، وخط قبل النص breadcrumb بتاع التقسيم (كتاب/باب/فصل). المادة الطويلة بس هي اللي تتقسم على الفقرات.

Key Findings

1) الملف غالباً ليه text layer نظيف، وفيه ناس اشتغلوا عليه قبل كده

- مشروع **ITI-Project** على GitHub (Mostafa-Ashraf-1) معمول على "egyptian_civil_code.pdf". الـ README بيوصفه كده: **170 صفحة**، ثنائي اللغة عربي/إنجليزي، "Text layer: Native text; OCR is not required". طلع منه **1,094 chunk** مبنيين على المواد، وكان بيحدد بداية المادة بكلمة "مادة" في العربي، وبيأخذ رقم المادة من "Article N" في الإنجليزي. الاستخراج كان بـ pypdf.
- مشروع **Egyptian-Legal-RAG-with-MLOPS** (eslam22elabd) بيقول إنه اتجرب على الـ PDF الحقيقي. بيستخدم get_text("blocks") من PyMuPDF، وبيقسم الـ blocks يمين وشمال حسب الـ x، ويعرف أنه عمود عربي بعدّ الحروف العربية فيه. ملاحظته الأهم إن العربي طلع **بترتيب حروف صح** بالطريقة دي، "unlike plain pdfplumber text extraction, which reversed Arabic-letter runs". وكمان بيّفك نطاقات المواد الملغاة لسجل مستقل لكل رقم عليه is_repealed=True.
- **تحذير:** المشروع الثاني بيقول إن العمودين "NOT guaranteed to align on the same page"، وعشان كده بيبيّن stream عالمي لكل لغة ويطلب بالترتيب. ده عكس اللي باين في الـ screenshots بتاعتك (صفوف جدول فيها الإنجليزي والعربي جنب بعض). ممكن يكونوا شغالين على نسخة ثانية من الملف، أو طريقة الـ blocks نفسها بتضيق حدود الصفوف. **الخلاصة إنك لازم تتأكد بنفسك** على 5–10 صفحات قبل ما تختار بين المطابقة بالصف والمطابقة بالترتيب. وفي كل الأحوال، رقم المادة هو المفتاح الحاسم.

(2) مقارنة أدوات الاستخراج

الأداة	العربي RTL	الجدول والصفوف المكمورة	التكلفة/السرعة	الحكم
PyMuPDF	الإصدارات من 1.24 وطالع بترجّع ترتيب منطقي، حسب Mahmoud Ibrahim (Confileo) على dev.to: "Below 1.24, Arabic comes back visual and reversed. From 1.24 onward issue it comes back logical" فيه متسجلة إن ligature "لا" ما بتتعمشش مع باقي الحروف في الإصدارات القديمة	<code>find_tables()</code> بيستخدم الخطوط المرسومة ("lines"), و <code>get_text(..., clip=cell)</code> بيطلع النص جوه كل خلية	مجاني، سريع جدًا (تواني 170J صفحة)، رخصة AGPL	الاختيار الأول
pdfplumber	فيه issues (#794 و #1159) إن <code>extract_tables / extract_text</code> بيطلعوا العربي مقلوب ومتلخبط. الإصدارات الجديدة ضافت <code>line_dir / char_dir</code> و <code>*_render</code> لدعم RTL	أحسن أداة لفحص الخطوط والـ <code>rects</code> والـ <code>words</code> بالإحداثيات	مجاني، أبسط من PyMuPDF	ممتاز للتشخيص والـ <code>debugging</code> ، ولازم تطبط <code>char_dir="rtl"</code>
pdfminer.six	هو اللي pdfplumber مبني عليه، وعنده نفس مشاكل الترتيب	مفيش API table	مجاني	ماتستخدموش مباشرة
camelot / tabula-py	مش معمولين للـ <code>tabula.py</code> ، شغالة بـ Java	كويسين في الجداول اللي خطوطها واضحة، بس مش هيعرفوا يكتلوا صف مكسور بين صفحتين	مجانين	مالهمش لازمة هنا
Docling	#253 issue عن PDF عربي: "the Arabic text is reversed (words in a sentence and letters per word)", و #3021 issue عن صعوبة الـ OCR العربي	layout كويس	مجاني، محتاج GPU/CPU تقيل	مش مضمون للعربي
MinerU	(2024) #677 issue: العربي مقلوب أو بيتقص كصورة. الإصدار 2.6.2 حدّث OCR العربي لـ <code>ppocr-v5</code> وقالوا الدقة زادت أكثر من 40%	cross-page table merge	مجاني، تقيل	جزّيه لو الملف scanned بس
Marker/Surya	Surya بيعمل OCR بس، مش reading order: "use something like marker... if you need to order the text". رخصة الأوزان معدّلة (مجانية) تحت \$5M	Surya عنده table recognition	GPU	OCR/J fallback
Azure AI Document Intelligence (Layout)	Layout و Read ضافوا العربي ضمن 42 لغة جديدة ووصلوا لـ 164 لغة	جداول + ترتيب قراءة + section headings	\$10.00 لكل 1,000 صفحة (S0، حسب DocuOCR من Azure Retail Prices API الرسمي)، يعني \$1.70 بالظبط للملف كله	أحسن خدمة cloud لو الملف scanned
Google Document AI	بيدعم العربي (ماتحققتش من الأرقام في البحث ده)	Layout parser	مدفوع	بديل لـ Azure
AWS Textract	مايبدعمش العربي. الـ docs الرسمية: "Amazon Textract supports English, Spanish, German Italian, French, and Portuguese"	—	—	استبعده

3) OCR لو الملف scanned

في **KITAB-Bench** (Heakl et al., MBZUAI, Findings of ACL 2025)، وفيه 8,809 samples across 9 major domains and 36 sub-domains، جدول OCR (image→text) نتايجه كالتالي: **0.13 Gemini-2.0-Flash CER**, 0.20 AIN-7B, 0.31 Tesseract, 0.54 EasyOCR, 0.79 PaddleOCR, 4.95 Surya. الورقة بتقول إن الـ VLMs "outperform traditional OCR approaches... by an average of 60% in Character Error Rate" recognition. وخذ بالك إن PaddleOCR تاريخيًا بيطلع العربي بالترتيب البصري (visual order)، فمحتاج bidi بعده.

4) Vision LLM: الدقة والهלוسة والتكلفة

- **التكلفة رخيصة جدًا.** الـ docs الرسمية لـ Gemini بتقول: "Each document page is equivalent to 258 tokens" في Gemini 2.x. يعني 170 صفحة ≈ 43,860 input token، ودي ≈ **\$0.013** على Gemini 2.5 Flash (input \$0.30/M, output \$2.50/M) هو اللي عليه أغلب التكلفة: حوالي \$0.43 لو ~1,000 token للصفحة. في Gemini 3.x الصفحة بتاخذ 560 token افتراضيًا، أو + 560 النص الـ native في وضع medium. Claude الصفحة "typically uses 1,500–3,000 tokens per page" نص، وفوقها تكلفة الصورة، يعني حوالي input \$1.5–2.7 على Sonnet 5 (\$10/2\$ للمليون). Haiku 4.5 محدود بـ 100 صفحة للـ request، فتهقسم الملف.
- **الخطر الحقيقي هو الأمانة مش الفلوس.** دراسة على وثائق تاريخية (2026) رصدت أربع أنماط أخطاء: "incomplete transcriptions, orthographic normalization, spurious additions, and semantic substitutions" "may alter the meaning of the source document while having only a limited impact on standard OCR metrics". OpenAI نفسها بتحذر في vision docs إن الموديل "may not perform optimally when handling images with text of non-Latin alphabets" (زي "يجوز" و"لا يجوز") معناها كارثة.
- **الخلاصة:** ماتخليش LLM هو مصدر النص. استخدمه **verifier/repairer**: تبعته صورة الصفحة + النص المستخرج وتسأل "فيه اختلاف؟ رقم المادة صح؟"، وأي اختلاف يترصد كـ flag، مايتصلحش أوتوماتيك.

Details

1) مشاكل العربي في الـ PDF وحلولها

1. **ترتيب مقلوب (visual order):** بعض الـ generators بتخزن الـ glyphs بترتيب الرسم. Mahmoud Ibrahim (Confileo) وثّق في بوست على dev.to عنوانه "The Arabic PDF bug was never in my code — it was the library version" إن PyMuPDF تحت 1.24 و 7.1 LibreOffice بيرجعوا ترتيب الرسم، وإن الإصدارات الأحدث بترجع ترتيب منطقي. وكتب: "the correct amount of bidi reconstruction code is zero... Everything I had written was scrambling text that had already arrived in the right order". **القاعدة:** ثبتت pymupdf>=1.24، واكشف الانعكاس الأول قبل ما تصلح أي حاجة.
2. **الكشف عن الانعكاس:** مؤشرات إحصائية بسيطة. في النص السليم كلمات كتير بتبدأ بـ "ال"، ومفيش كلمات بتبدأ بـ "ة" أو "ى". في النص المقلوب هتلاقى كلمات بتخلص بـ "لا" (يعني "ال" مقلوبة) وكلمات بتبدأ بـ "ة". كمان "مادة" هتظهر "ةدام". احسب score لكل خلية، ولو النص طالع مقلوب اعكسه **على مستوى الـ run العربي بس** وسيب الأرقام والنص اللاتيني بترتيبهم.
3. **Presentation Forms (U+FE70–FEFF, U+FB50–FDFF):** طبق `unicodedata.normalize("NFKC", s)` بعد ما تتأكد إن الترتيب منطقي. **ماتعملش NFKC على نص لسه visual.** مكتبة arabic-lint بتشرح ليه: "is one codepoint ligature: standing for two letters. NFKC expands it in logical order while the text around it is still in visual order" فالحرطين بيطلعوا متبديلين. والـ #5 issue عندهم بيقول إن فيه **465** شكل عربي الـ NFKC بيفكهم لأكثر من حرف، مش لام-ألف بس. **الترتيب الصح:** اصلح الاتجاه NFKC →. ولو مش ضامن، عمل NFKC على مستوى حرف بحرف بعد العكس.
4. **التطويل والتشكيل:** شيل التطويل (U+0640) دايمًا. التشكيل (U+064B–U+0652) احتفظ بيه في arabic_text الأصلي، وشيله من نسخة arabic_norm اللي بتستخدمها في البحث والـ embeddings. ممكن كمان تعمل normalization للهمزات (إ/أ/آ) في arabic_norm بس، لأن ده بيعتبر النص القانوني.

5. **الأرقام الهندية:** `str.maketrans("٠١٢٣٤٥٦٧٨٩٠١٢٣٤٥٦٧٨٩", "0123456789"*2)` . وخذ بالك إن pypdf كان عنده bug يقلب الأرقام الهندية (١٢٣٤→٤٣٢١)، واتصلّح في PR #4077. PR ده بيقول كمان إن "PyMuPDF extracts the same files correctly". **Validation مهم:** رقم مادة (٥٤) لو طلع 45 هيبان ك duplicate أو كسر في التسلسل، ودي فايدة تانية لفحص التسلسل.

6. **الأقواس المعكوسة:** في النص المقلوب ("٨٥") ممكن تطلع "٥٨". الـ regex بتاعك يقبل الشكلين `[()]\s*([0-9]+)\s*[()]` ، وبعد تصليح الاتجاه بدّل الأقواس لو محتاج.

7. **python-bidi / arabic-reshaper:** دول **للعرض بس** (terminal, matplotlib, رسم PDF). بيحوّلوا النص المنطقي لشكل بصري ويحطوا presentation forms، يعني عكس اللي إنت عاوزه للتخزين. **ماتستخدمش في pipeline الاستخراج أبدًا**، إلا لو عاوز تطبع العربي في terminal ما بيدعمش RTL.

ب) استراتيجية المطابقة (Alignment)

المستوى 1: الصف (row). الجدول ليه خطوط، فتقدر تستخدم `page.find_tables(strategy="lines")` تجيب بيه bbox لكل صف. الخلية الشمال إنجليزي والخلية اليمين عربي، وتحدد ده بعدّ الحروف العربية مش بالافتراض. لو `find_tables` فشل في صفحة، اكتشف الخط الرأسي الفاصل من `page.get_drawings()` : ده خط طوله قريب من ارتفاع الجدول، والـ x بتاعه قريب من نص الصفحة. والحدود الأفقية للصفوف خدها من الخطوط الأفقية.

المستوى 2: رقم المادة (المفتاح الحاسم).

- إنجليزي: `^\\s*Article\\s+(\\d+)\\b`
- عربي، بعد تحويل الأرقام: `^\\s*(?:(ال|م)دّة|\\s*(\\d+)\\s*[()])?\\s*(\\d+)\\s*[()])?`
- لو الصف فيه "Article 54" ومعها "Articles 54-80 have been repealed"، أو "المواد من ٥٤ إلى ٨٠ أُلغيت": regex للنطاق `Articles?\\s+(\\d+)\\s*[-]\\s*(\\d+).*repealed` و `Articles?\\s+(\\d+)\\s*إلى\\s+(\\d+)\\s*من\\s+المواد`.

المستوى 3: State machine بتمشي على الصفوف بالترتيب:

- **صف فيه header مادة:** افتح سجل جديد `current`.
- **صف مالوش header وماهوش عنوان تقسيم:** ده continuation، ضيف الإنجليزي على `current.english` والعربي على `current.arabic`، وحدّث `page_end`. ولو الإنجليزي بدأ مادة جديدة والعربي لسه continuation (أو العكس)، عالج كل لغة لوحدها: خلي لكل لغة `current` خاص بيها، واعمل merge بالرقم في الآخر. **ده أمتن تصميم**، ويحل كمان المشكلة اللي المشروع التاني اشتكى منها لو العمودين مش متزامنين.
- **صف عنوان تقسيم (BOOK/CHAPTER/SECTION...):** حدّث مكدس الـ hierarchy لكل لغة، وماتعبروش continuation.
- **النطاقات الملغاة:** عمل سجل لكل رقم (54...80) عليه `is_repealed=true`، والنص الأصلي في `repeal_note` و `repealed_range=[54,80]`. كده التسلسل والـ lookup بالرقم يفضلوا سليمين، وتقدر تستبعدهم من الـ `embeddings`.
- **قانون الإصدار:** خلي `scope = "issuing_law"` من أول الملف لحد ما تشوف أول عنوان للقانون نفسه ("SECTION I / الفصل الأول" أو "الباب التمهيدي")، أو لحد أول صف فيه "Article 1" إنجليزي. أي مادة عربي بس في الفترة دي تتسجل بمفتاح ("issuing_law", n)، ومواد القانون بمفتاح ("code", n). المفتاح المركّب ده بيمنع إن مادة الإصدار تتصادم مع Article 1.

المستوى 4: الـ hierarchy. عمل mapping صريح، لأن التسميات مابتقابلش حرفيًا:

مستوى	EN	AR
part	FIRST PART / PRELIMINARY	القسم الأول / الباب التمهيدي
book	BOOK I	الكتاب الأول
chapter	CHAPTER I	الباب الأول
section	SECTION I / Section I	الفصل الأول
subsection	1. Elements of Contracts	أولاً: أركان العقد

مستوى	EN	AR
topic	Consent / 2. Juristic persons	١- الرضاء / ٢- الشخص الاعتباري

ملاحظة: screenshots فيها " / SECTION I الفصل الأول " فوق " / FIRST PART القسم الأول "، وده غالبًا لأن المستويات متداخلة بطريقة مش خطية. **مايفترضش إن العمق ثابت**. لما يظهر عنوان من مستوى معين، امسح كل المستويات اللي تحته. ولو فيه لبس، خزن heading_path كقائمة بالترتيب اللي ظهرت بيه، بدل ما تحشرها في حقول ثابتة.

المستوى 5: الفقرات جوه المادة.

1. قسّم العربي على `^s*\((\d+)\)` بعد تحويل الأرقام.
2. قسّم الإنجليزي على سطور فاضية أو فواصل الفقرات (y-gap أكبر من ارتفاع السطر بكثير في PyMuPDF words).
3. لو العدد متساوي، طابق 1:1 وتأكد بالembedding.
4. لو مختلف، شغل **bertalign** (بيستخدم LaBSE) أو **vecalign** على جمل المادة دي بس. بما إن المادة صغيرة، الdynamic programming هيبقى دقيق وسريع. **AlignAR** (OSACT 2026) عمل alignment توليدي بالLLM على نصوص قانونية وأدبية عربي-إنجليزي، ووصل لـ "overall F1-score of 85.5%, a nearly 9% improvement over previous methods". ولقوا إن المجموعات السهلة (1:1، زي النصوص القانونية) مابتفرّقش بين الطرق كتير. يعني عندك الembedding aligners كفاية، والLLM يدخل في الحالات الصعبة بس.
5. **hunalign** و **bleualign** أقدم وأضعف من الطرق المبنية على الembeddings. ورقة "Evaluating automatic sentence alignment approaches on English-Slovak sentences" (Scientific Reports (2023) قارنت خمس أدوات (BSA) و **Hunalign** و **Bleualign** و **Vecalign** و **Bertalign**)، ولقت إن **Bertalign** و **Vecalign** هما significantly best and BSA the worst. **Hunalign** و **Bleualign** achieved the same performance in terms of F1 score."

Validation (ج)

- **التسلسل:** المواد في القانون من 1 لحد 1149. اطبع قائمة الأرقام الناقصة والمكررة لكل لغة لوحدها، وبعدين الأرقام الموجودة في لغة ومش موجودة في الثانية.
- **نسبة الطول:** `len(en_chars)/len(ar_chars)` لكل مادة. احسب median وMAD على الملف كله، وأي مادة بعيدة عن median بأكثر من 3×MAD تبقى flag. في أوراق ترشيح الparallel data، مدى بيتقال عليه `< |x_src|/|x_tgt| < 0.6` typically "1.7 كنقطة بداية، بس اضبطه على بياناتك (الإنجليزي عادة أطول من العربي بالحروف).
- **التشابه عبر اللغتين:** LaBSE أو bge-m3 على (en, ar) لكل مادة. IndicTrans2 استخدم LaBSE... with a cosine similarity threshold of 0.80. Artetxe & Schwenk نبّهوا إن "the scale of cosine similarity not being globally consistent across different sentences". **الأحسن** تعمل check نسي: cosine بين المادة N إنجليزي وN عربي لازم يبقى أعلى من N إنجليزي مع $1 \pm N$ عربي (margin). لو ماكانش كده، فيه غالبًا إزاحة مادة (off-by-one).
- **Spot-check بصري:** اختار 20 مادة عشوائي، ومعاهم كل المواد اللي عليها flag، وارسم الbboxes على صورة الصفحة بـ `page.get_pixmap()`، وراجعها بعينك.
- **highlights الصفراء:** دي annotations (`page.annots()`) مش جزء من النص، فمش هتأثر على الاستخراج. ممكن تسجلها في notes لو ليها معنى (تعديلات مثلاً).

Output schema (JSONL) (د)

```
{
  "id": "code-085",
  "scope": "code",
  "article_number": 85,
  "article_label_en": "Article 85",
  "article_label_ar": "مادة (٨٥)",
  "english_text": "...",
```

```

"arabic_text": "...",
"arabic_norm": "...",
"paragraphs": [
  {"idx": 1, "ar": "...", "en": "...", "align_score": 0.87}
],
"hierarchy_en": {"part": "FIRST PART", "book": "BOOK I", "chapter": "CHAPTER I", "section": "Section II",
"sub": ["1. Elements of Contracts", "Consent"]},
"hierarchy_ar": {"part": "القسم الأول", "book": "الكتاب الأول", "chapter": "الباب الأول", "section": "الفصل الثاني", "sub": ["أركان العقد: أولاً", "الرضاء ١-"]},
"breadcrumb_ar": "أركان العقد: أولاً > الفصل الثاني > الباب الأول > الكتاب الأول > القسم الأول",
"page_start": 12, "page_end": 13,
"is_repealed": false, "repealed_range": null, "repeal_note": null,
"alignment_confidence": 0.91,
"flags": [],
"notes": [],
"source": {"file": "egyption-low.pdf", "extractor": "pymupdf-1.26", "sha256": "..."}
}

```

PostgreSQL + pgvector: اعمل جدول `articles`، والـ PK عليه `(scope, article_number)`. اعمل كمان جدول `chunks` فيه: `metadata jsonb`، و `(bge-m3L) embedding vector(1024)`، `content`، `chunk_idx`، `lang`، `article_id` اعمل `embedding` لكل لغة لوحدها في صف لوحده. كده السؤال العربي يلاقى النص العربي والإنجليزي كمان لأن الموديل `multilingual` ولما ترجع النتيجة ترجع المادة كاملة باللغتين من جدول `articles`. استخدم `index` من نوع `HNSW` مع `vector_cosine_ops`، وضيف `full-text/trigram index` على `arabic_norm` عشان البحث برقم المادة أو بالكلمة بالطبط يبقى `hybrid`.

:Chunking

- مادة = `chunk`. أغلب مواد القانون المدني قصيرة.
- لو المادة طويلة (أكثر من ~400–500 token مثلاً)، قسّمها على حدود الفقرات (١)(٢). **ماتقسمش جوه فقرة أبداً**. وكل `chunk` يتكرر فيه سطر العنوان: "القانون المدني – مادة ٨٥ – [breadcrumb]".
- حط الـ `breadcrumb` في النص اللي بيتعمله `embed`، لأنه بيحسن الـ `retrieval` للأسئلة العامة زي "أحكام رهن الحيازة". مشروع ITI نفسه سجل فشل في السؤال ده بالطبط، وقال إن الـ `"retrieved an unrelated preamble article"` retrieval، ودي كمان حجة إنك تفصل قانون الإصدار بـ `scope`.
- المواد الملغاة: خزنها، لكن استبعدوها من الـ `index` أو خلي أولويتها قليلة.

(هـ) مصادر جاهزة ممكن توقّر عليك شغل

- **عربي:** `eastlaws.com` (نص كامل للقانون 131 لسنة 1948 مع مواد الإصدار)، `manshurat.org` ("وفقاً لأحدث تعديلاته – نسخة من إعداد وزارة المالية" بصيغة قابلة للتحميل)، `lawyeregypt.net`، وملف `doc` على `tantawylaw.com`. ملف الـ `doc` بالذات يقدر يدبك نص عربي منطقي من غير أي مشاكل PDF، وتستخدمه كـ **ground truth للمقارنة** مع استخراجك (بعد `diff` normalization).
- **إنجليزي:** ترجمة Andersen (Maher Milad Iskander & Co.) منشورة على Mondaq، وترجمة Perrott, Fanner & Sims (1952) Marshall، و WIPO Lex (نسخة موحدة بالتعديلات لحد "Law No. 55 of 1970"). وفيه كمان `arablawsworld.com` (مدفوعة).
- **Datasets:** على Hugging Face فيه `dataflare/egypt-legal-corpus` (25M+ token، عربي، MIT)، و `TawasuAI/egyptian-law-articles` و `fr3on/eg-legal-instruction-following` (4,184 مثال). **مالقيتش dataset ثنائي اللغة ومطابق على مستوى المادة للقانون المدني**، فالـ `output` بتاعك هيبقى إضافة فعلية.
- **تحذير:** النسخ دي مش بالضرورة نفس الصياغة أو نفس التعديلات اللي في ملفك، وحقوق الترجمات الإنجليزية مش واضحة. استخدمها للـ `validation`، مش كبديل من غير ما تراجع.

Recommendations: الـ pipeline المقترح

1. **Triage (5 دقائق):** شغل `page.get_text()` على أول 3 صفحات. لو رجع فاضي أو كلام ملخبط، الملف `scanned`، روح للخطوة 8. شغل كمان `detect_reversed()` على العربي.

2. **اكتشاف هيكل الجدول:** `find_tables()`، ولو فشل استخدم `get_drawings()`، وطلّع منها `bbox` لكل صف وخط الفصل الرأسّي.

3. استخراج كل خلية بـ `get_text("text", clip=cell_rect)`، وتحدد اللغة بنسبة الحروف العربية.

4. **Normalization:** إصلاح الاتجاه لو محتاج → NFKC → شيل التطويل → تحويل الأرقام → نسخة .arabic_norm

5. **State machine**: تصنيف الصف (عنوان / header مادة / / continuation نطاق ملغي / preamble)، مع current مستقل لكل لغة، ومفتاح (scope, n) .

6. **Merge بالرقم**، وبعددين alignment الفقرات، وبعددين validation (التسلسل، الطول، cosine margin).

7. **Export** JSON، ومعاه ملف review.csv للحالات اللى عليها flag، وبعددين load pgvector.

8. **Fallback:** لو الملف scanned أو في صفحات فشلت: Azure Layout (\$1.70 بالظبط على سعر \$10.00 لكل 1,000 صفحة) أو Gemini (أقل من دولار) لكل صفحة prompt بيطلب JSON فيه

عندك وقت، طابق الناتج مع ملف الـ doc العربي.

9. **verifierLLM اختياري:** للمواد التي عليها flag بس، ابعث الصورة + النصين واسأل عن الاختلافات. ماتقبلش التصليح الأوتوماتيكي.

Code skeleton

```
# pip install "pymupdf>=1.24" sentence-transformers
import re, json, unicodedata
from dataclasses import dataclass, field
import pymupdf # fitz

AR_DIGITS = str.maketrans("٠١٢٣٤٥٦٧٨٩٠١٢٣٤٥٦٧٨٩", "0123456789" * 2)
ARABIC_RE = re.compile(r"[\u0600-\u06FF\uFB50-\uFDFD\uFE70-\uFEFF]")
TATWEEL = "\u0640"
DIACRITICS = re.compile(r"[\u064B-\u0652\u0670]")

EN_ART = re.compile(r"^\s*Article\s+(\d+)\b", re.I)
EN_RANGE = re.compile(r"Articles?\s+(\d+)\s*[-]\s*(\d+).?*repealed", re.I | re.S)
AR_ART = re.compile(r"^\s*(?:ال|)؟ما دة\s*[\(\)]?\s*(\d+)\s*[\(\)]?")
AR_RANGE = re.compile(r"المواد\s+من\s+(\d+)\s+(\d+)\s+(?:الى|الى)\s+(\d+)\s+")

EN_HEAD = [{"part", r"^(FIRST|SECOND|PRELIMINARY)\s+PART\b"}, {"book", r"^BOOK\s+[IVXLC]+\b"}, {"chapter", r"^(CHAPTER\s+[IVXLC]+\b"}, {"section", r"^(SECTION\s+[IVXLC]+\b"}],
AR_HEAD = [{"part", r"^(القسم|التمهيد|الكتاب)\s+"}, {"book", r"^(الكتاب\s+"}, {"chapter", r"^(الباب\s+"}, {"section", r"^(الفصل\s+"}],
LEVELS = ["part", "book", "chapter", "section", "sub"]

def is_arabic(s): return len(ARABIC_RE.findall(s)) > 0.3 * max(1, len(s.replace(" ", "")))

def looks_reversed(s: str) -> bool:
    words = re.findall(r"[\u0621-\u064A]+", s)
    if len(words) < 5: return False
    fwd = sum(w.startswith("ال") for w in words)
    rev = sum(w.endswith("ى") for w in words) + sum(w[0] in "ةى" for w in words)
    return rev > fwd

def fix_arabic(s: str) -> str:
    if looks_reversed(s):
        # العربية بس، وسبب الأرقام واللاتيني بترتيبهم العكس ال
        s = "\n".join(" ".join(reversed([w[::-1] if ARABIC_RE.search(w) else w
                                         for w in line.split()]))) for line in s.splitlines())
    s = unicodedata.normalize("NFKC", s).replace(TATWEEL, "")
    return s.translate(AR_DIGITS)

def ar_norm(s): # بس للـ embeddings وال
    s = DIACRITICS.sub("", s)
    return re.sub("[\u064B\u0670]", "ا", s).replace("ى", "ي")
```

```

@dataclass
class Rec:
    scope: str; number: int; lang_text: dict = field(default_factory=lambda: {"en": "", "ar": ""})
    pages: list = field(default_factory=list); hier: dict = field(default_factory=dict)
    is_repealed: bool = False; repealed_range: list | None = None; flags: list = field(default_factory=list)

def table_rows(page):
    tabs = page.find_tables(strategy="lines")
    for t in tabs:
        for row in t.rows:
            cells = [c for c in row.cells if c]
            yield [pymupdf.Rect(c) + (-2, -2, 2, 2) for c in cells]

def extract(path):
    doc = pymupdf.open(path)
    stacks = {"en": {}, "ar": {}}
    cur = {"en": None, "ar": None}
    recs = {} # (scope, n) -> Rec
    scope = "issuing_law"
    for pno, page in enumerate(doc, 1):
        for cells in table_rows(page):
            texts = [page.get_text("text", clip=r).strip() for r in cells]
            en = next((t for t in texts if t and not is_arabic(t)), "")
            ar = fix_arabic(next((t for t in texts if t and is_arabic(t)), ""))
            for lang, txt, heads, art, rng in (("en", en, EN_HEAD, EN_ART, EN_RANGE),
                                              ("ar", ar, AR_HEAD, AR_ART, AR_RANGE)):
                if not txt: continue
                first = txt.splitlines()[0].strip()
                lvl = next((l for l, p in heads if re.match(p, first, re.I)), None)
                if lvl: # عنوان تقسيم
                    if lang == "en" or lvl in ("part", "chapter"): scope = "code"
                    st = stacks[lang]; st[lvl] = first
                    for deeper in LEVELS[LEVELS.index(lvl) + 1:]: st.pop(deeper, None)
                    continue
                m_r, m_a = rng.search(txt), art.match(txt)
                if lang == "en" and m_a: scope = "code"
                if m_r: # نطاق ملغي
                    a, b = map(int, m_r.groups())
                    for n in range(a, b + 1):
                        r = recs.setdefault((scope, n), Rec(scope, n))
                        r.is_repealed, r.repealed_range = True, [a, b]
                        r.lang_text[lang] = txt; r.pages.append(pno)
                    cur[lang] = None; continue
                if m_a: # مادة جديدة
                    n = int(m_a.group(1))
                    r = recs.setdefault((scope, n), Rec(scope, n))
                    if r.lang_text[lang]: r.flags.append(f"duplicate_{lang}")
                    r.lang_text[lang] = txt; r.pages.append(pno)
                    r.hier[lang] = dict(stacks[lang]); cur[lang] = r
                elif cur[lang]: # continuation
                    cur[lang].lang_text[lang] += "\n" + txt; cur[lang].pages.append(pno)
                else:
                    recs.setdefault(("orphan", pno), Rec("orphan", pno)).lang_text[lang] += txt
    return recs

def validate(recs, model=None):
    code = sorted(n for (s, n) in recs if s == "code")
    missing = sorted(set(range(1, max(code) + 1)) - set(code))
    ratios = {}
    for k, r in recs.items():
        en, ar = r.lang_text["en"], r.lang_text["ar"]
        if not en: r.flags.append("missing_en")
        if not ar: r.flags.append("missing_ar")
        if en and ar: ratios[k] = len(en) / len(ar)
    if model: # LaBSE / bge-m3: الحد الجيران margin فحص الـ
        keys = [k for k in sorted(ratios) if k[0] == "code"]
        E = model.encode([recs[k].lang_text["en"] for k in keys], normalize_embeddings=True)
        A = model.encode([recs[k].lang_text["ar"] for k in keys], normalize_embeddings=True)
        S = E @ A.T
        for i, k in enumerate(keys):
            nb = [S[i, j] for j in (i - 1, i + 1) if 0 <= j < len(keys)]

```

```

        if S[i, i] < 0.6 or (nb and S[i, i] <= max(nb)):
            recs[k].flags.append(f"low_align:{S[i, i]:.2f}")
        return missing

def export(recs, out="civil_code.jsonl"):
    with open(out, "w", encoding="utf-8") as f:
        for (scope, n), r in sorted(recs.items(), key=lambda x: (x[0][0] != "issuing_law", x[0][1])):
            f.write(json.dumps({
                "id": f"{scope}-{n:04d}", "scope": scope, "article_number": n,
                "english_text": r.lang_text["en"], "arabic_text": r.lang_text["ar"],
                "arabic_norm": ar_norm(r.lang_text["ar"]),
                "hierarchy_en": r.hier.get("en"), "hierarchy_ar": r.hier.get("ar"),
                "page_start": min(r.pages, default=None), "page_end": max(r.pages, default=None),
                "is_repealed": r.is_repealed, "repealed_range": r.repealed_range,
                "flags": r.flags, ensure_ascii=False) + "\n")

if __name__ == "__main__":
    recs = extract("egyption-low.pdf")
    # from sentence_transformers import SentenceTransformer; m = SentenceTransformer("sentence-transformers/
    LaBSE")
    print("missing:", validate(recs))
    export(recs)

```

ملاحظات على الكود:

- العتبات (margin, 0.6) مجرد نقطة بداية، اضبطها بعد ما تشوف توزيع الـ scores.
- row.cells ممكن يرجع None للخلايا المدمجة (merged)، وده اللي بيحصل غالبًا في صفوف العناوين اللي ماشية بعرض الجدول كله. عالجهما بإنك تاخذ النص كله وتقسمة حسب الـ x بتاع الخط الرأسى.
- الـ continuation اللي في أول صفحة بتتضاف لآخر مادة مفتوحة في نفس اللغة، وده بيحل مشكلة كسر الصفحة من غير أي heuristics على الـ y.

Caveats

- ماشفتش الملف نفسه. الكلام عن إن فيه text layer نظيف وإن PyMuPDF بيطلع ترتيب صح جاي من README مشروعين على GitHub بيوصفوا ملف مشابه (170 صفحة، ثنائي اللغة). ومفيش ضمان إنه نفس ملفك بالظبط.
- الكلام إن PyMuPDF من 1.24 بيطلع ترتيب منطقي جاي من تجربة Mahmoud Ibrahim (Confileo) في بوست على dev.to ("The Arabic PDF bug was never in my code – it was the library version")، مش من الـ changelog الرسمي. اختبره على ملفك.
- **تحديث بعد التجربة:** جربت PyMuPDF 1.28 على PDF تجريبي ثنائي اللغة اتعمل بـ LibreOffice، ولقيته بيقلب الأرقام الهندية («مادة (٨٥)» طلعت (٥٨))، وبيووظ «لا» («الأول» طلعت «أول»)، ويحط الفاصلة في أول السطر. يعني الترتيب المنطقي مش مضمون حتى في الإصدارات الجديدة. الأضمن ترتيب الحروف حسب مكانها على الصفحة، وده اللي بيعمله rtl_safe_extract.py.
- أرقام KITAB-Bench مقاسة على benchmark عام فيه أنواع وثائق كتير، مش على نص قانوني مطبوع نظيف. على صفحة واضحة زي دي، الفرق بين الأدوات هيبقى أقل.
- سعر Azure Layout (\$10.00 لكل 1,000 صفحة) جاي من DocuOCR، اللي سحبته من Azure Retail Prices API الرسمي (ساري من 2021-08-01)، وأسعار GPT-4o/4.1 جاية من مصادر ثانية غير الصفحات الرسمية، لأن الصفحات الرسمية ماتحملتش. أسعار Gemini و Claude من الـ docs الرسمية (سبتمبر 2026)، ويمكن تتغير. وصفحة Gemini API deprecations الرسمية بتقول إن الوصول لموديلات 2.5 بقى محدود لليوزر اللي استخدموها قبل كده، وبتنصح بـ "3.5 Flash-Lite or 3.8 Flash" لأي مشروع جديد. وصفحة Google Cloud model lifecycle حاطة يوم 20 أكتوبر 2026 كتاريخ retirement لـ gemini-2.5-flash حسب Zentor.
- عتبة cosine 0.80 جاية من ترشيح corpora عامة. النص القانوني بمصطلحاته الخاصة ممكن يطلع scores أقل حتى لو الترجمة صح، فاعتمد على الـ margin النسبي أكثر من العتبة المطلقة.

• الترجمات الإنجليزية المنشورة (WIPO، Andersen، إلخ) ممكن تختلف في الصياغة وفي التعديلات عن اللي في ملفك، وحقوق استخدامهما مش واضحة.

المصادر

مشاريع على نفس الملف

1. <https://github.com/Mostafa-Ashraf-1/ITI-Project> — Mostafa-Ashraf-1/ITI-Project
2. <https://github.com/eslam22elabd/Egyptian-Legal-RAG-with-MLOPS> — eslam22elabd/Egyptian-Legal-RAG-with-MLOPS

أدوات الاستخراج والعربي في الـ PDF

1. <https://github.com/pymupdf/PyMuPDF/issues/2199> — PyMuPDF issue #2199: RTL (Arabic) ligatures
2. <https://github.com/pymupdf/PyMuPDF/discussions/> — PyMuPDF discussion #3768: find_tables cell words 3768
3. <https://pymupdf.readthedocs.io/en/latest/page.html> — PyMuPDF docs: Page
4. <https://www.nutrient.io/blog/extract-text-from-pdf-> — Nutrient: Python PDF text extraction with PyMuPDF pymupdf/
5. <https://github.com/jsvine/pdfplumber/issues/794> — pdfplumber issue #794: Wrong Ordering of RTL Text
6. <https://github.com/jsvine/pdfplumber/issues/> — pdfplumber issue #1159: Arabic reversed text in tables 1159
7. <https://github.com/jsvine/pdfplumber/blob/stable/CHANGELOG.md> — pdfplumber CHANGELOG
8. <https://github.com/DS4SD/docling/issues/253> — Docling issue #253: Support for right to left languages
9. <https://github.com/docling-project/docling/issues/3021> — Docling issue #3021: Scanned Arabic text
10. <https://github.com/opencv/opencv/issues/677> — opencv/opencv issue #677: Arabic support
11. <https://opendatalab.github.io/MinerU/reference/changelog/> — MinerU changelog
12. <https://github.com/datalab-to/surya> — Surya (datalab-to)
13. <https://docs.azure.cn/en-us/ai-services/document-intelligence/reference/release-history?view=doc-intel-4.0.0> — Azure Document Intelligence release history
14. <https://docs.azure.cn/en-us/ai-services/document-intelligence/whats-new?view=doc-intel-4.0.0> — Azure Document Intelligence: What's new
15. <https://docs.aws.amazon.com/textract/latest/dg/textract-best-practices.html> — Amazon Textract best practices
16. <https://invoicedataextraction.com/blog/> — Invoice Data Extraction: Python OCR library for Arabic tables python-ocr-library-arabic-table
17. <https://dev.to/> — dev.to (Confileo): Why Arabic text comes out backwards from a PDF support_confileo_ce7442eb/why-arabic-text-comes-out-backwards-when-you-extract-it-from-a-pdf-and-how-to-fix-it-548g
18. https://github.com/nada19885/Legal_Case/ — nada19885/Legal_Case PR #3: Arabic stored in visual order pull/3
19. <https://pypi.org/project/arabic-lint/0.7.0/> — arabic-lint (PyPI)
20. <https://github.com/Syamjith-NK/arabic-lint/issues/5> — arabic-lint issue #5: multi-codepoint forms

<https://github.com/py-pdf/pypdf/pull/4077> — pypdf PR #4077: Arabic-Indic digits reversed .21

<https://github.com/6f5/rtl-arabic> — 6f5/rtl-arabic .22

الـ OCR و Vision LLMs

<https://github.com/mbzuai-oryx/KITAB-Bench> — KITAB-Bench (GitHub) .1

<https://arxiv.org/abs/2502.14949> — KITAB-Bench (arXiv 2502.14949) .2

<https://ai.google.dev/gemini-api/docs/document-processing> — Gemini API: Document understanding .3

<https://ai.google.dev/gemini-api/docs/interactions/media-resolution.md.txt> — Gemini API: Media resolution .4

<https://aicostcheck.com/blog/google-gemini-pricing-guide-2026> — AI Cost Check: Gemini API pricing 2026 .5

<https://platform.claude.com/docs/en/build-with-claude/pdf-support> — Claude docs: PDF support .6

<https://platform.claude.com/docs/en/about-claude/pricing.md> — Claude docs: Pricing .7

<https://arxiv.org/html/2607.24077v3> — When Low CER is Not Enough (arXiv) .8

<https://developers.openai.com/api/docs/guides/images-vision> — OpenAI API: Images and vision .9

المطابقة والـ Validation

<https://github.com/bfsujason/bertalign> — bertalign .1

<https://aclanthology.org/2026.osact-1.7/> — AlignAR (OSACT 2026, ACL Anthology) .2

<https://arxiv.org/pdf/2601.10310> — arXiv 2601.10310 .3

<https://arxiv.org/pdf/2305.16307> — IndicTrans2 (arXiv 2305.16307) .4

<https://arxiv.org/pdf/1811.01136> — Margin-based Parallel Corpus Mining (arXiv 1811.01136) .5

نصوص القانون والـ Datasets

<https://www.eastlaws.com/legislation-full-text/en/egypt/law/29-07-1948/no-131?type=1&id=2123159> — Eastlaws: Law No. 131 of 1948 (English) .1

<https://www.eastlaws.com/legislation-full-text/ar/egypt/law/29-07-1948/no-131?type=1&id=384> — Eastlaws: القانون رقم 131 لسنة 1948 (عربي) .2

<https://manshurat.org/node/72413> — منشورات قانونية: القانون المدني وفقا لأحدث تعديلاته .3

<https://www.tantawylaw.com/images/2.doc> — Tantawy Law (.doc) .4

<https://lawyeregypt.net/%D8%A7%D9%84%D9%85%D9%83%D8%AA%D8%A8%D8%A9-%D8%A7%D9%84%D9%82%D8%A7%D9%86%D9%88%D9%86%D9%8A%D8%A9/%D9%86%D8%B5%D9%88%D8%B5-%D8%A7%D9%84%D9%82%D8%A7%D9%86%D9%88%D9%86-%D8%A7%D9%84%D9%85%D8%AF%D9%86%D9%89-%D8%A7%D9%84%D9%85%D8%B5%D8%B1%D9%89-%D8%B1%D9%82%D9%85-131-%D9%84%D8%B3%D9%86%D8%A9-1948/> — Lawyer Egypt: نصوص القانون المدني .5

<https://www.mondaq.com/civil-law/1655638/translation-of-law-no-131-of-1948> — Mondaq: Translation of Law No. 131 of 1948 .6

<https://lawcat.berkeley.edu/record/376017?ln=en> — Berkeley Law: The Egyptian Civil Code .7

<https://www.wipo.int/wipolex/en/legislation/details/8362> — WIPO Lex: Civil Code, Egypt .8

- <https://www.arablawsworld.com/egypt-laws/all/egypt-civil-code> — Arab Laws Online: Egyptian Civil Code in English .9
- <https://huggingface.co/datasets/dataflare/egypt-legal-corpus> — Hugging Face: dataflare/egypt-legal-corpus .10
- <https://huggingface.co/datasets/fr3on/eg-legal-instruction-following/blob/main/README.md> — Hugging Face: fr3on/eg-legal-instruction-following .11